

TP2-Raphaël

Code server.py

```
import socket
import threading
import time

# --- CONFIGURATION ---
HOST = '127.0.0.1'
PORT = 65432
NB_JOUEURS = 2 # Le jeu ne démarre que quand 2 clients sont là

# Chargement des questions
def load_questions(filename):
    questions = []
    try:
        with open(filename, 'r', encoding='utf-8') as f:
            for line in f:
                parts = line.strip().split(';')
                if len(parts) == 6:
                    # Format: Question, RepA, RepB, RepC, RepD, BonneReponse
                    questions.append({
                        "q": parts[0],
                        "options": parts[1:5],
                        "answer": parts[5]
                    })
    except FileNotFoundError:
        print(f"Erreur : Fichier {filename} introuvable.")
    return questions

QUESTIONS = load_questions("questions.txt")

# La barrière de synchronisation
# Elle bloquera les threads tant que 'NB_JOUEURS' n'ont pas atteint le point de rendez-vous
barrier = threading.Barrier(NB_JOUEURS)

def client_handler(conn, addr, player_id):
    print(f"[JOUEUR {player_id}] Connecté depuis {addr}")
    score = 0

    conn.send(f"Bienvenue Joueur {player_id}. En attente des autres joueurs...".encode('utf-8'))

    # 1. Attente que tous les joueurs soient connectés avant de commencer
    try:
        barrier.wait()
    except threading.BrokenBarrierError:
        return
```

```

conn.send("START".encode('utf-8'))

# 2. Boucle du Quiz
for i, q in enumerate(QUESTIONS):
    # Préparation du message question
    msg = f"\n--- QUESTION {i+1} ---\n{q['q']}\nA: {q['options'][0]}\nB: {q['options'][1]}\nC: {q['options'][2]}\nD: {q['options'][3]}\nVotre réponse (A/B/C/D) ?"
    conn.send(msg.encode('utf-8'))

    # Réception de la réponse
    try:
        reponse = conn.recv(1024).decode('utf-8').strip().upper()
    except ConnectionError:
        break

    # Vérification
    is_correct = (reponse == q['answer'])
    if is_correct:
        score += 1
        res_msg = "Correct !"
    else:
        res_msg = f"Faux ! La bonne réponse était {q['answer']}."

    conn.send(f"{res_msg} (Score actuel: {score})".encode('utf-8'))

    print(f"[JOUEUR {player_id}] Q{i+1}: Réponse {reponse} -> Score {score}")

# 3. SYNCHRONISATION : On attend que TOUS les joueurs aient fini cette question
conn.send("\nAttente des autres joueurs...".encode('utf-8'))
try:
    barrier.wait() # Tout le monde s'attend ici
except threading.BrokenBarrierError:
    break

# Fin du jeu
final_msg = f"\n--- FIN DU QUIZ ---\nVotre score final : {score}/{len(QUESTIONS)}"
conn.send(final_msg.encode('utf-8'))

# Sauvegarde du score (optionnel demandé en TP1)
with open("scores.txt", "a") as f:
    f.write(f"Joueur {player_id} ({addr}): {score}/{len(QUESTIONS)}\n")

conn.close()

def main():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind((HOST, PORT))
    server.listen(NB_JOUEURS)
    print(f"Serveur de Quiz démarré sur {HOST}:{PORT}")
    print(f"En attente de {NB_JOUEURS} joueurs...")

    threads = []

    # Accepter exactement NB_JOUEURS connexions
    for i in range(NB_JOUEURS):

```

```

        conn, addr = server.accept()
        t = threading.Thread(target=client_handler, args=(conn, addr, i+1))
        t.start()
        threads.append(t)

    # Attendre la fin de tous les threads (fin du jeu)
    for t in threads:
        t.join()

    print("Partie terminée. Fermeture du serveur.")
    server.close()

if __name__ == "__main__":
    main()

```

client.py

```

import socket

HOST = '127.0.0.1'
PORT = 65432

def main():
    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    try:
        client.connect((HOST, PORT))
    except ConnectionRefusedError:
        print("Impossible de se connecter au serveur (est-il lancé ?)")
        return

    print("Connecté au serveur de Quiz !")

    while True:
        try:
            # Réception du message serveur (Question ou Résultat)
            msg = client.recv(4096).decode('utf-8')
            if not msg:
                break

            print(msg)

            # Si le message attend une réponse (contient "Votre réponse")
            if "Votre réponse" in msg:
                reponse = input("-> ")
                client.send(reponse.encode('utf-8'))

            # Détection de fin de jeu
            if "FIN DU QUIZ" in msg:
                break

        except Exception as e:
            print(f"Erreur : {e}")
            break

```

```
client.close()
print("Connexion fermée.")

if __name__ == "__main__":
    main()
```

Questions.txt :

Quelle est l'unité de la tension électrique ?;Volt;Ampère;Ohm;Watt;A

Que signifie HTTP ?;High Transfer;HyperText Transfer Protocol;Html
Type;Home Tool;B

Quel port est utilisé par défaut pour le Web (HTTP) ?;21;22;80;443;C

Scores.txt :

Joueur 1 (('127.0.0.1', 17570)): 1/3

Joueur 2 (('127.0.0.1', 48994)): 2/3